

Enabling High-Goodput LLM Inference Serving with Heterogeneous SLO-Aware Speculative Decoding

Paper #1664

Background and Motivation

Modern *large language model* (LLM) [17, 18, 25] serving is becoming shared infrastructure for applications with different latency requirements [5]. Interactive dialogue [14, 19] requires a steady token stream, code completion [4, 13] demands rapid generation for seamless workflows, while document processing [1, 20] and complex reasoning [22, 23, 29] can tolerate more relaxed generation rates, prioritizing quality over immediacy. The increasing multiplexing of these workloads on shared GPU clusters demands that LLM serving systems support heterogeneous *service-level objectives* (SLOs) [6, 15, 28], rather than imposing a uniform generation target.

For streaming LLM services, responsiveness after the first token is largely governed by *time per output token* (TPOT), which captures the sustained decoding rate of a request. Heterogeneous TPOT SLOs shift the serving objective from maximizing throughput to allocating decoding resources under SLO constraints. Continuous batching [8, 24] remains essential for improving GPU utilization, but conventional batching policies typically advance all requests uniformly at each decoding iteration, providing limited control over each request’s progress toward its individual TPOT target, as illustrated in Figure 1(a). When requests with tight and relaxed TPOT SLOs are batched together, such uniform progress can overallocate decoding work to requests with ample latency slack, while requests close to SLO violations receive insufficient priority. As a result, throughput no longer reflects the effective serving capacity under heterogeneous SLOs. The optimization objective should instead be **SLO-constrained goodput** [28], which counts only the tokens generated by requests that meet their assigned TPOT SLOs [26].

Speculative decoding [2, 3, 9, 12, 21] provides a promising mechanism for adaptive progress control under heterogeneous SLOs. It uses a lightweight draft model to propose multiple draft tokens and then verifies them with the target model in one iteration, allowing each request to yield a variable number of accepted tokens per decoding step, as illustrated in Figure 1(b). For a single request, its sustained generation latency can be approximated as $TPOT \approx \frac{T_{\text{draft}} + T_{\text{verify}}}{\mathbb{E}[A]}$, where T_{draft} and T_{verify} are the draft and verification latencies, and $\mathbb{E}[A]$ is the expected number of accepted tokens per speculative iteration. This relation reveals a tunable tradeoff governed by the *speculative budget*, i.e., the number of draft tokens proposed and verified in each speculative iteration. A larger budget can improve $\mathbb{E}[A]$, but also increases draft and

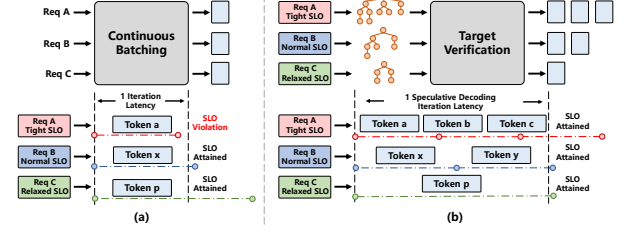


Figure 1. Motivation for SLO-aware speculative serving.

verification latency. This tradeoff naturally supports SLO-aware control, where requests with tight TPOT SLOs receive larger budgets to accelerate progress, while requests with relaxed SLOs receive smaller budgets to conserve computation.

Limitations of Current Approaches

Existing SLO-aware speculative serving systems [7, 10] exploit this tradeoff by assigning different budgets to requests with different latency targets. However, they retain a serial draft-then-verify pattern where the next draft stage must wait for verification to finish. Consequently, aggressive budgets directly increase the draft latency exposed in each iteration. This serial coupling becomes more severe in batched serving, where concurrent requests share compute resources and progress under synchronization boundaries. If the total batch-level speculative capacity is fixed to bound iteration latency, each request receives a smaller effective budget as the batch grows, reducing accepted progress per round. If each request keeps its own budget, the total number of draft and verification candidates grows with batch size, substantially increasing iteration latency for all requests. For AdaServe, as the active batch size grows from 8 to 64, the mean number of accepted tokens per request drops from 4.1 to 1.7, confirming this latency-progress dilemma.

Recent asynchronous parallel speculative decoding systems [11, 27] attempt to hide draft latency by overlapping drafting with target verification. However, this hidden drafting window is susceptible to prefix invalidation because draft tokens generated from an unconfirmed prefix are useful only if the target model fully accepts that prefix. Otherwise, the continuation is discarded. For PEARL with draft length 4, only 32.4% of hidden draft tokens are accepted, while 38.4% are invalidated by prefix rejection and 29.2% are rejected during verification. More importantly, existing parallel mechanisms optimize overlap efficiency rather than allocate the hidden drafting window based on TPOT urgency. As batch size grows, tight-SLO attainment drops sharply while relaxed requests remain mostly satisfied. This indicates that

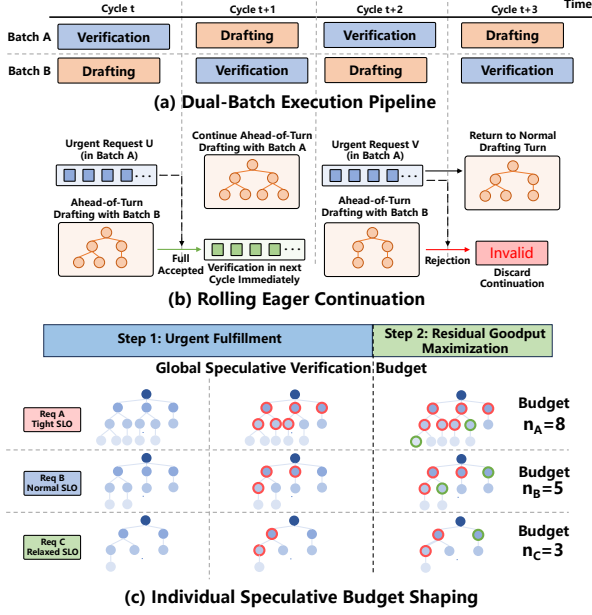


Figure 2. Overview of SpecRhythm.

the hidden drafting capacity is not directed to the requests that most need timely progress.

Key Ideas and Design

We present **SpecRhythm**, a speculative LLM serving system designed to maximize goodput. The key idea is to separate the execution structure that creates speculative capacity from the control policy that allocates that capacity. At the execution layer, SpecRhythm builds a stable batch-level overlap structure and treats the resulting drafting windows as allocatable resources. At the control layer, it decides which requests should draft ahead of their normal batch turn and how much speculative budget each request should receive.

SpecRhythm realizes this design through three integrated techniques. First, the **Dual-Batch Execution Pipeline** organizes active requests into two alternating logical batches. In each service step, the target model verifies the draft tokens of one batch, while the draft model generates new draft tokens for the other batch. In the next step, the two batches swap roles. This structure hides drafting work within the complementary verification window and provides a predictable batch-level drafting window for scheduling control.

Second, **Rolling Eager Continuation** exploits the available overlap window to advance urgent requests before their scheduled batch turn. Standard dual-batch rotation forces requests to wait until their assigned drafting stage, which lacks the flexibility needed for those near their SLO boundary. SpecRhythm evaluates progress gaps right before target verification. For highly urgent requests, it initiates ahead-of-turn drafting by injecting them into the active drafting window of the alternating batch. If the current prefix is accepted, these eagerly drafted tokens immediately serve as ready candidates; otherwise, they are discarded to preserve

correctness. By hiding draft latency, this technique accelerates token generation for latency-critical requests.

Third, **Individual Speculative Budget Shaping** determines how candidate tokens are distributed under a bounded verification budget. Target verification becomes more expensive as the number of candidate tokens grows, so SpecRhythm uses offline profiling to identify a roofline-guided verification budget that keeps verification within a low-latency region. Within this budget, SpecRhythm first allocates candidates to urgent requests with projected progress gaps, prioritizing high-confidence tokens that are more likely to be accepted. Any remaining candidate slots are then assigned to high-confidence candidates across requests, weighted by residual urgency. This two-step policy avoids wasting speculative capacity on requests with sufficient slack or low expected acceptance, while still improving overall goodput.

Experimental Results

We implement SpecRhythm on top of nano-vLLM and evaluate it on a single node with 4× NVIDIA A100 80GB GPUs, pairing the Llama3.1-70B-Instruct and Qwen3-32B target models with the Llama3.2-1B-Instruct and Qwen3-0.6B draft models, respectively. Baselines include SpecInfer (serial speculative decoding) [12], AdaServe (SLO-aware serial speculative decoding) [10], and PEARL (asynchronous parallel speculative decoding) [11]. The default workload mixes tight (code completion [4]), normal (chatbot [16]), and relaxed (summarization [1]) requests at a 6:2:2 ratio.

Across diverse workloads, SpecRhythm consistently improves goodput and SLO attainment. At an active batch size of 64, it delivers 721.3 tokens/s with 80.7% attainment on Llama, and 764.8 tokens/s with 76.4% attainment on Qwen3, outperforming AdaServe by up to 1.27× in goodput and 1.31× in attainment. Under dynamic online traffic at 4 RPS, SpecRhythm sustains 603.3 tokens/s on Llama and 711.0 tokens/s on Qwen3 while maintaining the highest SLO attainment among all baselines. Similar gains hold under stricter SLO targets and higher fractions of tight requests, demonstrating that SpecRhythm remains robust as heterogeneous SLO pressure increases. An ablation study further shows that overlap alone is insufficient. On Llama3, adding only the dual-batch pipeline reduces goodput from 182.4 to 150.4 tokens/s, while rolling eager continuation and budget shaping raise it to 392.5 and 723.0 tokens/s. The same trend holds on Qwen3, where goodput changes from 168.0 to 130.6, 358.5, and 577.8 tokens/s across the same cumulative variants.

Summary

Overall, SpecRhythm advances heterogeneous LLM serving with a batch-level SLO-aware speculative decoding design. A dual-batch execution pipeline creates predictable drafting capacity, while rolling eager continuation and individual budget shaping allocate it by runtime urgency and draft confidence to improve goodput and SLO attainment.

References

- [1] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. 2024. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*. 3119–3137.
- [2] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. 5209–5235.
- [3] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318* (2023).
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [5] Jiangfei Duan, Runyu Lu, Haojie Duanmu, Xiuhong Li, Xingcheng Zhang, Dahua Lin, Ion Stoica, and Hao Zhang. 2024. MuxServe: flexible spatial-temporal multiplexing for multiple LLM serving. *arXiv preprint arXiv:2404.02015* (2024).
- [6] Ke Hong, Xiuhong Li, Lufang Chen, Qiuli Mao, Guohao Dai, Xuefei Ning, Shengen Yan, Yun Liang, and Yu Wang. 2025. Sola: Optimizing slo attainment for large language model serving with state-aware scheduling. *Proceedings of Machine Learning and Systems* 7 (2025).
- [7] Kaiyu Huang, Hao Wu, Zhuo Shi, Han Zou, Minchen Yu, and Qingjiang Shi. 2025. AdaSpec: Adaptive Speculative Decoding for Fast, SLO-Aware Large Language Model Serving. In *Proceedings of the 2025 ACM Symposium on Cloud Computing*. 361–374.
- [8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles (SOSP 23)*. 611–626.
- [9] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EA-GL: Speculative Sampling Requires Rethinking Feature Uncertainty. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. 28935–28948.
- [10] Zikun Li, Zhuofu Chen, Remi Delacourt, Gabriele Oliaro, Zeyu Wang, Qinghan Chen, Shuhuai Lin, April Yang, Zhihao Zhang, Zhuoming Chen, Yi-Hsiang Lai, Xinhao Cheng, Xupeng Miao, and Zhihao Jia. 2026. AdaServe: Accelerating Multi-SLO LLM Serving with SLO-Customized Speculative Decoding. In *Proceedings of the 21st European Conference on Computer Systems, EuroSys (EuroSys 26)*. 36–54.
- [11] Tianyu Liu, Yun Li, Qitan Lv, Kai Liu, Jianchen Zhu, Winston Hu, and Xiao Sun. 2025. PEARL: Parallel Speculative Decoding with Adaptive Draft Length. In *Proceedings of the 13th International Conference on Learning Representations (ICLR 25)*.
- [12] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. 2024. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 24)*. 932–949.
- [13] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474* (2022).
- [14] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- [15] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *Proceedings of the 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA 24)*. 118–132.
- [16] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Stanford alpaca: An instruction-following llama model.
- [17] Llama Team. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).
- [18] Qwen Team. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).
- [19] Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, YaGuang Li, Hongrae Lee, Huaixiu Steven Zheng, Amin Ghafouri, Marcelo Menegali, Yanping Huang, Maxim Krikun, Dmitry Lepikhin, James Qin, Dehao Chen, Yuanzhong Xu, Zhifeng Chen, Adam Roberts, Maarten Bosma, Yanqi Zhou, Chung-Ching Chang, Igor Krivokon, Will Rusch, Marc Pickett, Kathleen S. Meier-Hellstern, Meredith Ringel Morris, Tulsee Doshi, Renelito Delos Santos, Toju Duke, Johnny Soraker, Ben Zevenbergen, Vinodkumar Prabhakaran, Mark Diaz, Ben Hutchinson, Kristen Olson, Alejandra Molina, Erin Hoffman-John, Josh Lee, Lora Aroyo, Ravi Rajakumar, Alena Butryna, Matthew Lamm, Viktoriya Kuzmina, Joe Fenton, Aaron Cohen, Rachel Bernstein, Ray Kurzweil, Blaise Agüera y Arcas, Claire Cui, Marian Croak, Ed H. Chi, and Quoc Le. 2022. LaMDA: Language Models for Dialog Applications. *arXiv preprint arXiv:2201.08239* (2022).
- [20] Dongsheng Wang, Natraj Raman, Mathieu Sibue, Zhiqiang Ma, Petr Babkin, Simerjot Kaur, Yulong Pei, Armineh Nourbakhsh, and Xiaomo Liu. 2024. Docllm: A layout-aware generative language model for multimodal document understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 8529–8548.
- [21] Runze Wang, Qinggang Wang, Haifeng Liu, Long Zheng, Xiaofei Liao, Hai Jin, and Jingling Xue. 2026. Adaptive Draft Sequence Length: Enhancing Speculative Decoding Throughput on PIM-Enabled Systems. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1–15.
- [22] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).
- [23] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [24] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
- [25] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. OPT: Open Pre-trained Transformer Language Models. *arXiv preprint arXiv:2205.01068* (2022).
- [26] Wei Zhang, Zhiyu Wu, Yi Mu, Rui Ning, Banruo Liu, Nikhil Sarda, Myungjin Lee, and Fan Lai. 2026. {JITServe}:{SLO-aware}{LLM} Serving with Imprecise Request Information. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*.

- 825–848.
- [27] Ziyi Zhang, Ziheng Jiang, Chengquan Jiang, Menghan Yu, Size Zheng, Haibin Lin, Xin Liu, and Henry Hoffmann. 2026. SwiftSpec: Disaggregated Speculative Decoding and Fused Kernels for Low-Latency LLM Inference. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 26)*. 2197–2211.
- [28] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.
- [29] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, et al. 2022. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625* (2022).